

The program looks secure, but is not. The problem is that the time of the access check for privileges and the time at which the privileges are used are not the same. Assume that a fraction of a second after the check by *access*, the attacker manages to create a symbolic link with the same file name to the password file. In that case, the *open* will open the wrong file, and the write of the attacker's data will end up in the password file. To pull it off, the attacker has to race with the program to create the symbolic link at exactly the right time.

The attack is known as a **TOCTOU (Time of Check to Time of Use)** attack. Another way of looking at this particular attack is to observe that the *access* system call is simply not safe. It would be much better to open the file first, and then check the permissions using the file descriptor instead—using the *fstat* function. File descriptors are safe, because they cannot be changed by the attacker between the *fstat* and *write* calls. It shows that designing a good API for operating system is extremely important and fairly hard. In this case, the designers got it wrong.

9.5.9 Double Fetch Vulnerability

A race condition very similar to TOCTOU occurs when the kernel fetches data from user processes twice. Consider a system call that takes a buffer from a user process (to send over the network, write to a file, or output to a printer). To copy the buffer into its own address space, the kernel first reads the length field from an address in the user process and allocates its own buffer of that size. Next, it uses the value at that same memory location again to copy the content of the user buffer into the newly allocated kernel buffer. What could possibly go wrong?

Having seen TOCTOUs, you quickly realize that the answer is a race condition where another thread modifies the length field between the allocation and the copy operation. By making it larger, an attacker can cause a buffer overflow.

A well-known example of a TOCTOU-like double fetch vulnerability was found in Windows, where untrusted software is subjected to security checks before it is allowed to perform sensitive operations. For instance, the security software in Windows would modify the entries of a table that contains the addresses of (potentially sensitive) services that a program may call directly. By replacing these addresses with those of its own functions, the security software ensures that its own functions are always executed first. These functions perform some checks on the parameters and then call the original Windows services. This technique is known as **hooking**. Unfortunately, by calling the services first with parameters that pass the checks and then modifying the parameters to malicious values just before they are used, attackers could bypass the checks.

9.6 EXPLOITING HARDWARE

Just like software, hardware may contain vulnerabilities also. For a long time, security experts dismissed such vulnerabilities as impractical and complicated to exploit, but that attitude changed in a hurry when a new class of vulnerabilities was

disclosed in 2018 and everyone, from hardware vendors to operating system developers, got their knickers in a twist. The vulnerabilities were given the apocalyptic names of Meltdown and Spectre and prominently featured in the news. Since then, security researchers have found tens of variants of these vulnerabilities. The implications for operating systems are severe. To discuss all of them, we would need another book, but we will only look at the main underlying issues. To do so, we must first explain about covert and side channels. If you are interested in more detail about Meltdown and Spectre, see Lipp et al. (2020) and Amit et al. (2021).

9.6.1 Covert Channels

In Sec. 9.3, we discussed formal models of secure systems. All these ideas about formal models, cryptography, and provably secure systems sound great, but do they actually work? In a word: No. Even in a system which has a proper security model underlying it and which has been proven to be secure and is correctly implemented, security leaks can still occur. In this section, we discuss how information can still leak out even when it has been rigorously proven that such leakage is mathematically impossible. These ideas are due to Lampson (1973).

Lampson's model was originally formulated in terms of a single timesharing system, but the same ideas can be adapted to LANs and other multiuser environments, including applications running in the cloud. In the purest form, it involves three processes on some protected machine. The first process, the client, wants some work performed by the second one, the server. The client and the server do not entirely trust each other. For example, the server's job is to help clients with filling out their tax forms. The clients are worried that the server will secretly record their financial data, for example, maintaining a secret list of who earns how much, and then selling the list. The server is worried that the clients will try to steal the valuable tax program.

The third process is the collaborator, which is conspiring with the server to indeed steal the client's confidential data. The collaborator and server are typically owned by the same person. These three processes are shown in Fig. 9-23. The object of this exercise is to design a system in which it is impossible for the server process to leak to the collaborator process the information that it has legitimately received from the client process. Lampson called this the **confinement problem**.

From the system designer's point of view, the goal is to encapsulate or confine the server in such a way that it cannot pass information to the collaborator. Using a protection-matrix scheme we can easily guarantee that the server cannot communicate with the collaborator by writing a file to which the collaborator has read access. We can probably also ensure that the server cannot communicate with the collaborator using the system's interprocess communication mechanism.

Unfortunately, more subtle communication channels may also be available. For example, the server can try to communicate a binary bit stream as follows. To send a 1 bit, it computes as hard as it can for a fixed interval of time. To send a 0 bit, it goes to sleep for the same length of time.

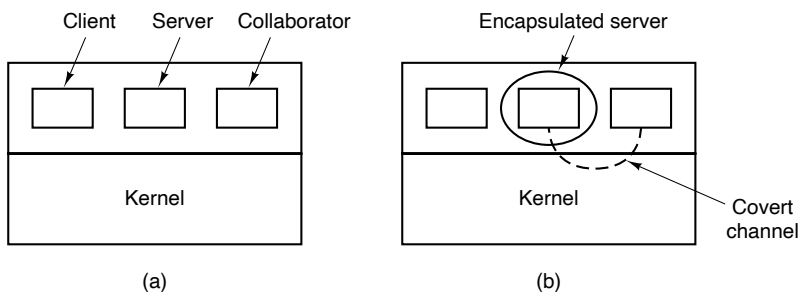


Figure 9-23. (a) The client, server, and collaborator processes. (b) The encapsulated server can still leak to the collaborator via covert channels.

The collaborator can try to detect the bit stream by carefully monitoring its response time. In general, it will get better response when the server is sending a 0 than when the server is sending a 1. This communication channel is known as a **covert channel**, and is illustrated in Fig. 9-23(b).

Of course, the covert channel is a noisy channel, containing a lot of extraneous information, but information can be reliably sent over a noisy channel by using an error-correcting code (e.g., a Hamming code, or even something more sophisticated). The use of an error-correcting code reduces the already low bandwidth of the covert channel even more, but it still may be enough to leak substantial information. It is fairly obvious that no protection model based on a matrix of objects and domains is going to prevent this kind of leakage.

Modulating the CPU usage is not the only covert channel. The paging rate can also be modulated (many page faults for a 1, no page faults for a 0). In fact, almost any way of degrading system performance in a clocked way is a candidate. If the system provides a way of locking files, then the server can lock some file to indicate a 1, and unlock it to indicate a 0. On some systems, it may be possible for a process to detect the status of a lock even on a file that it cannot access. This covert channel is illustrated in Fig. 9-24, with the file locked or unlocked for some fixed time interval known to both the server and collaborator. In this example, the secret bit stream 11010100 is being transmitted.

Locking and unlocking a prearranged file, *S*, is not an especially noisy channel, but it does require fairly accurate timing unless the bit rate is very low. The reliability and performance can be increased even more using an acknowledgement protocol. This protocol uses two more files, *F1* and *F2*, locked by the server and collaborator, respectively, to keep the two processes synchronized. After the server locks or unlocks *S*, it flips the lock status of *F1* to indicate that a bit has been sent. As soon as the collaborator has read out the bit, it flips *F2*'s lock status to tell the server it is ready for another bit and waits until *F1* is flipped again to indicate that another bit is present in *S*. Since timing is no longer involved, this protocol is fully

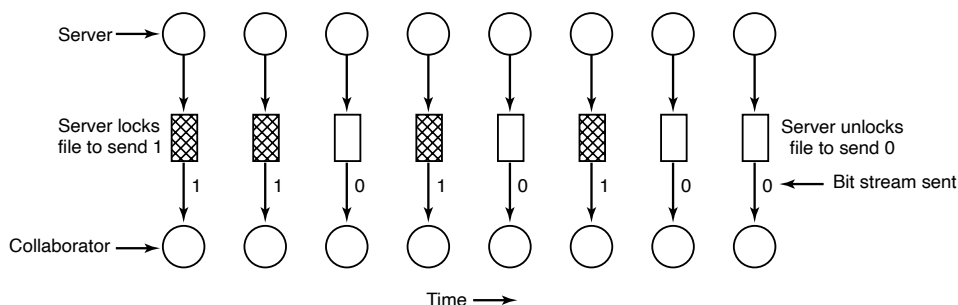


Figure 9-24. A covert channel using file locking.

reliable, even in a busy system, and can proceed as fast as the two processes can get scheduled. To get higher bandwidth, why not use two files per bit time, or make it a byte-wide channel with eight signaling files, *S0* through *S7*?

Acquiring and releasing dedicated resources (tape drives, plotters, etc.) can also be used for signaling. The server acquires the resource to send a 1 and releases it to send a 0. In UNIX, the server could create a file to indicate a 1 and remove it to indicate a 0; the collaborator could use the `access` system call to see if the file exists. This call works even though the collaborator has no permission to use the file. Unfortunately, many other covert channels exist.

Lampson also mentioned a way of leaking information to the (human) owner of the server process. Presumably the server process will be entitled to tell its owner how much work it did on behalf of the client, so the client can be billed. If the actual computing bill is, say, \$100 and the client's income is \$53,000, the server could report the bill as \$100.53 to its owner.

Just finding all the covert channels, let alone blocking them, is nearly hopeless. Introducing a process that causes page faults at random or otherwise spends its time degrading system performance in order to reduce the bandwidth of the covert channels is not an attractive idea.

In the next section, we introduce a particularly insidious covert channel, based on hardware properties. In some ways, it is worse than the ones above, because it can also be used to steal sensitive information—a trick known as a side channel.

9.6.2 Side Channels

So far, we have assumed that two parties, a sender and a receiver, use a covert channel to transmit sensitive information, deliberately. However, sometimes, we can use similar techniques to leak information from a victim process without the victim's knowledge. In this case, we speak of **side channels**. Often a channel can function as a covert channel or as a side channel, depending on how it is used.

A good example is the cache side channel. Like all covert and side channels, it depends on a shared resource, in this case the cache. Suppose Andy receives a nice collection of “zebra and tree” pictures over a secure messenger tool. The tool encrypts all messages for a particular receiver with that receiver’s secret key before transmitting it (or delivering it to another user on the same computer). At the destination, the messages are stored in encrypted form and can only be read by the user with that same key. Suppose also that Herbert, another user on the same machine, is interested in Andy’s messages (and especially the zebra pictures). He can dump the contents of the messages on disk, but since they are encrypted, all they contain is garbage. If only he had that blasted key!

The messenger tool uses a well-known encryption routine *Encrypt()* from a shared crypto library. Like many encryption routines, it iterates over the bits in the secret key, doing one thing if the bit is a 0 and another thing if the bit is 1. See Fig. 9-25.

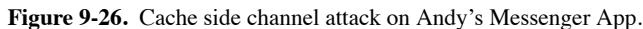
```
for (i = 0; i < length (SecretKey); i++)  
    if (SecretKey[i] == 0) do_one_thing (message, ...);  
    else do_another_thing (message, ...);
```

Figure 9-25. Structure of an encryption routine which iterates over the bits in the key, taking different actions depending on the value of the bit.

We do not care about the details of the encryption routine (which probably involves the kind of clever math that makes your head spin). What is important here is that the code that is executed when the key bit is 0 is at a different location than the code that is executed when the bit is 1 (see also Fig. 9-26, top). When the locations in memory are different, these instructions will also be placed at different locations in the cache. In other words, if Herbert can determine which location in the cache is used during each iteration, he also knows the value of that bit of the key.

Unfortunately (for Herbert), this sounds difficult: the cache does not simply tell you which cachelines are used when. Nevertheless, this information can be still be observed *indirectly*. The property that we use is that accessing something that is in the cache is fast, while accessing something that is not yet in the cache takes considerably longer. In this example, we will assume that the cache is used for both code and data (e.g., the level 3 cache on Intel processors), but similar attacks are possible for other caches also.

To discover the key, Herbert runs a program that also uses the crypto library and that continuously flushes the cachelines corresponding to the code of *do_one_thing()* and *do_another_thing()* from the cache (e.g., using the *clflush* instruction on x86 processors) and then immediately reads them back from memory, each time measuring very accurately how long these reads take. See also Fig. 9-26. While this program is running and timing the reads, he sends a message to Andy so that the messenger tool will encrypt it with Andy’s key.



In this way, he obtains Andy’s secret key bit by bit without ever touching it directly. This particular cache side channel is known as **Flush & Reload**. There

are other cache side channels also, but however interesting they may be, the details are beyond the scope of this book. Clearly, the cache can also be used for covert channels: by agreeing that accessing one cacheline means 0 and another cacheline means 1, a sender and receiver can exchange arbitrary messages. We will see later how novel and frankly fairly scary attacks use cache-based covert channels to leak sensitive information from the operating system kernel.

You may wonder what Andy could have done to thwart Herbert's cowardly and indirect attacks on his co-author's cryptographic key. One answer here is: use better software. For instance, by carefully designing your encryption routine to be **constant time**, with no observable timing differences between the different values for the key bit, the side channel no longer works. For instance, assume a new design of *Encrypt()* where *do_one_thing()* and *do_one_thing()* use the same cache lines. In that case, Herbert's code will not be able to use the above side channel to distinguish between different cases.

9.6.3 Transient Execution Attacks

In January 2018, the Meltdown and Spectre hardware vulnerabilities became public and Intel, one of the affected CPU vendors, saw its stock price drop by several percentage points. The world looked on in astonishment. It suddenly dawned that it could no longer trust the hardware. Moreover, the vendors indicated that some of the issues would not be fixed. What was going on?

The new vulnerabilities consisted of hardware vulnerabilities that could be exploited from software. Before we dive into the details, we should mention that these are very advanced attacks that are keeping security researchers and operating system developers busy around the world. They are also very cool.

Since Meltdown and Spectre, researchers have found many new members of that family of vulnerabilities. They are all based on optimizations in the CPU that make sure that the CPU is kept as busy as possible, so that it does that not waste time waiting. The way they achieve this is by making the CPU perform operations ahead of schedule.

In Sec. 5.1, we looked at one such optimization whereby instructions that were started later would start and often finish well before an earlier instruction finishes execution. For instance, *DIV* (division) is an expensive instruction. It gets worse if an operand has to be fetched from memory and is not in the cache. After the CPU has started such an instruction, it may take many clock cycles before it completes. What is the CPU to do in the meantime? Since most CPUs are superscalar, they have many execution units. For instance, they have multiple units to load values from memory, multiple units to perform integer addition and subtraction, etc. If the instruction following the division is an addition that does not depend on the result of the division, there is no harm in execution that ahead-of-time. And the next instruction. And the next. By the time the division is finally done, many later instructions will have completed already and all their results can now be committed.

Of course, the CPU has to be careful with such **out-of-order execution**. If something goes wrong during the execution of the *DIV*, for instance if the divisor turns out to be 0, it must raise an exception and undo all the effects of the subsequent out-of-order instructions. In other words, it must make it so that it appears as if the out-of-order instructions were never executed. These instructions are *transient*.

The problem is that such **transient execution** is squashed only at the architectural level—the level that is visible to the program(mers). So, the CPU makes sure that the transient instructions will have had no effect on registers or memory. However, there may still be traces of the executed-and-then-squashed code at the *micro-architectural* level. The micro-architecture is what implements a particular instruction set architecture. It includes the sizes and replacement policies of the caches, the TLB, the execution units, etc. For example, many CPUs from different companies implement the x86-64 instruction set architecture and they can all run the same programs, but under the hood, at the micro-architectural level, these CPUs are very different. Transient execution that is squashed at the architectural level may leave traces at the micro-architectural level, for instance because data that was loaded from memory is now in the cache.

How bad is that, you ask? Well, as we saw in the previous section, the presence or absence of data in the cache may be used as a covert channel. And that is exactly what is happening in these attacks.

Transient Execution Attacks Based on Faults

For efficiency, operating systems such as Linux map the operating system kernel into the address space of every user process. Doing so makes system calls cheaper, because even though the system call causes a switch to a more privileged domain, the kernel, there is no need to change the page tables. To make sure that user processes cannot modify kernel pages, the pages table entries for kernel memory have the Supervisor bit set (see Fig. 3.11).

Now consider the code in Fig. 9-27, where an unprivileged attacker tries to read a kernel address in Line 2. The CPU will not be enthusiastic about the idea, because the supervisor bit is set for that page, so the instruction will fault and throw an exception. This will happen when the corresponding instruction is retired. In the meantime, however, the CPU will continue under the assumption that all is well. It will (transiently) read the value and (transiently) execute the instruction in Line 3, which uses it as an index in an array. When the exception is finally raised, the architectural effects of the instructions are reverted. For instance, when the dust settles, the original values will be in *reg0* and *reg1*. The problem is that the transient instruction in Line 3 still had an effect on the micro-architectural state, as *array[reg0 * 4096]* is now in the cache. See also Fig. 9-28. Before executing these three instructions, the attacker makes sure that none of the other elements are in the cache. This is easy: just access a lot of other data, so that all the cachelines


```

1. char *kaddr = ...           // a kernel address
2. reg0 = kaddr[0]             // read byte from kernel address: not allowed
3. reg1 = array[reg0 * 4096]    // (transiently) use the value as an index

```

Figure 9-27. Meltdown: a user access kernel memory and uses it as an index.

are used for other stuff. This means, that after executing the above code, there is exactly one array element in the cache. By reading every array element (in steps of 4096) and measuring how long it takes to do the access, the attacker will find that one array element is considerably faster than the others. If the fast read occurred for the array element at, say, offset $7 * 4096$, the attacker knows that the secret byte read from the kernel was 7. In this way, attackers can leak each and every byte in the operating system kernel. Not a pleasant thought!

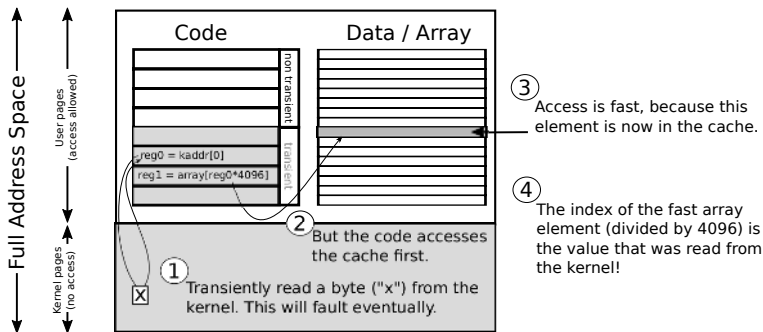


Figure 9-28. A value read by a faulting instruction still leaves a trace in the cache.

If you are curious about the multiplication with 4096, this is a common trick. Since a cache line is 64B on most architectures, if the attack had used the value in `reg0` as an index by itself without multiplication, the same cacheline would be used for all the values from 0 to 7. While the attacker could have used a different value for the multiplication, the factor 4096 ensures that each byte value lands on a unique cache line (and that related loads by the CPU's prefetcher do not matter).

The attack is known as **Meltdown** and led to a major change in operating system design. The fact that the Linux kernel developers originally proposed to call their patch "Forcefully Unmap Complete Kernel With Interrupt Trampolines," while flaunting their proclivity for clever acronyms, suggests that they were not entirely filled with joy about the achievements of chip vendors. Another suggested name was "User Address Space Separation," another gem. Eventually, the solution came to be known as **KPTI (Kernel Page Table Isolation)**. By completely separating the address spaces of the kernel and the user processes and giving the kernel its own set of page tables, it was no longer possible to leak kernel information. However, the cost in performance is significant. In newer processors, Meltdown is fixed in silicon but that does not help users with old hardware.

So far the good news. The bad news is that related vulnerabilities still pop up from time to time. They have different names and the details always vary, but the principle remains that a faulting instruction raises an exception, but in the meantime transient execution has already accessed and used the secret data.

Transient Execution Attacks Based on Speculation

There is another cause of transient execution: speculation. Consider the code in Fig. 9-29. Assume that the code runs inside the victim (for instance, the operating system) and that *input* is an unsigned integer value that is provided by an untrusted user process. Clearly the programmer has tried to do the right thing. Before using *input* as an index into an array, the program verifies that it is in bounds. Only if this is the case will it execute Lines 2 and 3. At least, that is what you would think.

```
1. if (input < MaxArrayElements) { // security check: do not allow buffer overflow?
2.   char x = A[input];           // read a char from the array
3.   char y = B[x * 4096];        // use the result as an index
4. }
```

Figure 9-29. Speculative execution: the CPU mispredicts the condition in Line 1 and executes Lines 2–3 speculatively, accessing memory that should be off-limits.

Reality is very different and the CPU may decide to execute the instructions *transiently* even if the index is out of bounds. The reason may be that the condition in Line 1 takes a long time to resolve. For instance, variable *MaxArrayElements* may not be in the cache and that means that the processor has to fetch it all the way from main memory, an operation that takes many cycles. In the meantime, the CPU with all its execution units has nothing to do. Stalling the CPU for such long periods would be disastrous for performance, so the hardware vendors came up with a clever trick. They said: what if we try to *guess* the outcome of the if condition? Or better still, can we somehow *predict* this value? If we predict the condition to be true, we can then execute the instructions in Lines 2–3 *speculatively* while waiting for the actual outcome of the condition to be resolved. The prediction is often based on history. For instance, if the outcome was true the last 100 times, it will probably be true again the 101st time. In reality, branch predictors in modern CPUs are much more complicated and very accurate.

Suppose we predicted the value to be true and speculatively executed the other two instructions. At some point the true outcome of the condition becomes available. If we guessed right and the prediction matched the actual outcome, we already have the results of the next two instructions and the CPU can simply commit them and move on to the next instruction. In case the prediction was wrong, no harm is done—we simply do not commit the results and undo all architecturally visible effects of these instructions. Since the speculatively executed instructions are now made transient, it will be as if they never executed.

Except that we already saw in the previous section that there may still be traces at the micro-architectural level, for instance in the cache. For simplicity's sake, let us assume that array *B* is shared between the attacker process and the victim (see also Fig. 9-30). This is not a strict requirement and the attack is still possible if the attacker cannot access the array directly, but a shared array simplifies the explanation. In particular, just like in the previous section, the attacker can just read all the elements of array *B* in a loop, while timing the access durations. If the access of element *n* is considerably slower, the attacker knows that *n*/4096 must have been the value that was accessed transiently.

What makes this especially dangerous is that the attacker may *train* the CPU's predictor to mispredict. For instance, the attacker can provide 100 inputs that are in-bounds to trick the branch predictor into thinking that the outcome will be true the 101st time also. However, this time the attacker provides an illegal, out-of-bounds-value to read data from a location that should not be accessible.

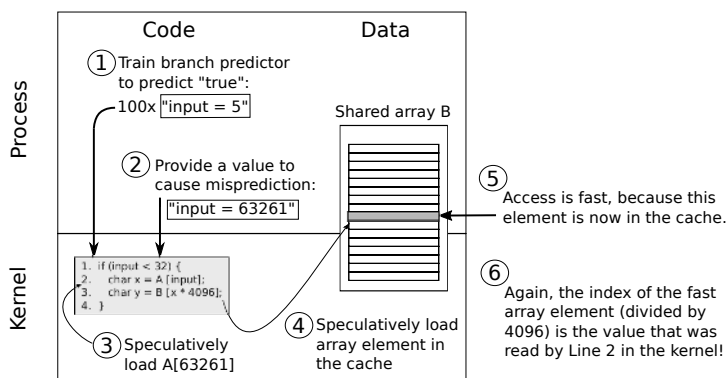


Figure 9-30. Original Spectre attack on the kernel.

By repeating the process, each time using the cache side channel to leak a new secret byte, the attacker can “read” out the victim’s entire address space, byte by byte. Even if the speculatively executed instructions access invalid memory locations, it does not matter, as transient execution does not crash. It will simply squash the result and resume execution at the appropriate location.

The attack is known as **Spectre**. There are many variants of speculative execution attacks. The example in this section is known as Spectre Variant 1. Mitigations against speculative execution attacks are even more problematic than in the case of Meltdown and it may be that some variants of Spectre will never be fixed. The reason is that speculative execution is so important for performance. Even so, mitigations for different variants exist both in software and hardware and executing a Spectre attack on modern processors and modern operating systems is not trivial. As an example, the variant we showed in this section can be mitigated in software

by inserting a memory fence (such as the one we saw in Sec. 2.5.9) right after the branch instruction in Line 1, which simply stops all speculation until the condition is resolved.

Transient execution attacks such as Meltdown and Spectre have spawned a whole new domain of security research. New vulnerabilities are found every few months and new fixes are released by CPU vendors and operating system developers at the same frequency. Unfortunately, all these mitigations chip away at performance. You may find that some newer processors (with all defenses up) are slower than older processors. We have not seen that often in the past 50 years!

9.7 INSIDER ATTACKS

A whole different category of attacks are what might be termed “inside jobs.” These are executed by programmers and other employees of the company running the computer to be protected or making critical software. These attacks differ from external attacks because the insiders have specialized knowledge and access that outsiders do not have. Below we will give a few examples; all of them have occurred repeatedly in the past. Each one has a different flavor in terms of who is doing the attacking, who is being attacked, and what the attacker is trying to achieve.

9.7.1 Logic Bombs

In these times of massive outsourcing, programmers often worry about their jobs. Sometimes they even take steps to make their potential (involuntary) departure less painful. For those who are inclined toward blackmail, one strategy is to write a **logic bomb**. This device is a piece of code written by one of a company’s (currently employed) programmers and secretly inserted into the production system. As long as the programmer feeds it its daily password, it is happy and does nothing. However, if the programmer is suddenly fired and physically removed from the premises without warning, the next day (or next week) the logic bomb does not get fed its daily password, so it goes off. Many variants on this theme are also possible. In one famous case, the logic bomb checked the payroll. If the personnel number of the programmer did not appear in it for two consecutive payroll periods, it went off (Spafford et al., 1989).

Going off might involve clearing the disk, erasing files at random, carefully making hard-to-detect changes to key programs, or encrypting essential files. In the latter case, the company has a tough choice about whether to call the police (which may or may not result in a conviction many months later but certainly does not restore the missing files) or to give in to the blackmail and rehire the ex-programmer as a “consultant” for an astronomical sum to fix the problem (and hope that he does not plant new logic bombs while doing so).